

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Machine learning

Progress report 4

Comparative Study of Machine Learning and Deep Learning for Hate Speech Classification

Advisor(s): Trương Vĩnh Lân

Student(s): Trần Quốc Bảo Long ID 2252453

Nguyễn Bình Nguyên ID 2252545

Đặng Ngọc Phú ID 2252617

Đặng Duy Nguyên ID 2352821

Nguyễn Văn Hoàng Phát ID 2352896

HO CHI MINH CITY, APRIL 2026



Contents

1	Project Overview	3
2	Error Analysis: Where Does the Best Model Fail?	3
2.1	Methodology	3
2.2	Error Rates	4
2.3	Representative Error Examples	4
2.4	Failure Mode Taxonomy	6
3	Model Interpretability	7
3.1	Methodology	7
3.2	Top Discriminative Features	7
3.3	Discussion	9
4	Class Imbalance: SMOTE vs Class Weighting	10
4.1	Methodology Recap	10
4.2	Results	10
4.3	Discussion	11
5	Submission Readiness	12
5.1	Checklist	12
5.2	Notebook Colab Compatibility	14
5.3	GitHub Repository (Bonus +5%)	14
5.4	Task Allocation	14
5.5	Final Report Integration Plan	15
6	Conclusion	16



1 Project Overview

This report documents the work completed in **Step 3** of the assignment: extended analysis and submission preparation. While Reports 2 and 3 covered the required pipeline (EDA, preprocessing, feature extraction, model training, hyperparameter tuning, and a deep learning comparison), the present report addresses the depth-of-analysis dimension of the rubric (**Quality of analysis and experiments — 25%**) and the submission-readiness criteria.

The work is organised as follows:

1. **Error analysis** of the best model — examining *where* and *why* predictions fail, with a categorisation of failure modes.
2. **Model interpretability** — extracting the most discriminative features from the best traditional classifier (LinearSVC + TF-IDF), showing which n-grams the model learns to associate with toxicity.
3. **Imbalance strategy comparison** — quantifying the gap between SMOTE over-sampling and class-weight balancing.
4. **Submission readiness** — Colab-compatibility, GitHub repository plan, task allocation table, and final-report integration plan.

2 Error Analysis: Where Does the Best Model Fail?

2.1 Methodology

The fine-tuned DistilBERT model (overall F1-macro = 0.871) is examined on the test set ($n = 6,393$). Predictions are split into:

- **False Negatives (FN):** actual label = toxic, predicted = non-toxic. These are toxic tweets the model failed to detect — the most costly error type for content-moderation use cases.
- **False Positives (FP):** actual label = non-toxic, predicted = toxic. These are benign tweets incorrectly flagged.

For each error class, the most-confidently-wrong predictions (sorted by predicted probability of the wrong class) are inspected manually to identify recurring patterns. The same



procedure is applied to the LinearSVC + TF-IDF model for cross-validation of failure modes.

2.2 Error Rates

Table 2.1 shows the per-class error rates of the best traditional and deep-learning models.

Table 2.1: Error rates on the test set ($n_{\text{toxic}} = 448$, $n_{\text{non-toxic}} = 5,945$)

Model	TN	FP	FN	TP
LinearSVC + TF-IDF	5,734	211	114	334
DistilBERT (fine-tuned)	5,845	100	67	381

DistilBERT halves the false-positive count ($211 \rightarrow 100$) and reduces false negatives by $\sim 40\%$ ($114 \rightarrow 67$). LinearSVC misses 25.4% of toxic tweets versus DistilBERT’s 15.0%. For both models, false positives are roughly twice as numerous as false negatives in absolute terms — but proportionally, the model is far worse at *detecting* toxicity (15–25% miss rate) than *avoiding* false alarms (1.7–3.5% rate).

2.3 Representative Error Examples

Table 2.2 and Table 2.3 present a selection of the most-confidently-wrong predictions made by LinearSVC + TF-IDF (cleaned text shown). These reveal systematic blind spots that are largely shared by DistilBERT, although DistilBERT recovers a fraction of cases through contextual reasoning.



Table 2.2: Selected false negatives — toxic tweets the model missed

#	Cleaned text (truncated)	Pattern
1	blackpeople beat whiteperson bp might make cou alive...	Coded violence
2	bluelivesmatter woke manspreading okay ifyourenotwhiteyourenotracist	Political dog-whistle
3	based recent commercial state farm old navy omarosa choice clearly...	Implicit aggression
4	rode journalist integrity greed ambition along total absence courage	Insult through accusation
5	potus attempting sabotage country final day israel obamacare isi energy...	Nationalist insinuation
6	running hater bitch whiteknight copyright copyright copyright...	Slurs masked by token repetition
7	smileyong along kingconrad make go hell	Direct attack with unfamiliar names

Table 2.3: Selected false positives — non-toxic tweets the model flagged

#	Cleaned text (truncated)	Pattern
1	bigot brigade still pumping klanservative racebait	Counter-speech
2	slip threat america bigotry misogyny ignorance alwaystrup	Anti-bigotry tweet
3	anyone suppos clinton feminist basis care white centered feminism...	Discussion of feminism
4	yes republican voter polled believe trump said judge curiel racist...	Reporting on racism
5	problem even realize mentioning judge ethnicity racist	Meta-commentary on racism
6	pathetic attempt invoke dead war hero defend one many trump racist	Critique of racist behaviour



2.4 Failure Mode Taxonomy

Inspection of all 114 false negatives and 211 false positives produced by LinearSVC + TF-IDF yields the following taxonomy.

False-negative patterns (toxic missed).

- **Implicit / coded language** — toxicity expressed through suggestion or comparison rather than slurs. Statements like “*blackpeople beat whiteperson bp might make cou alive...*” encode threats but contain no individually toxic tokens.
- **Political dog-whistles** — phrases such as “*ifyourenotwhiteyourenotracist*” and “*bluelivesmatter manspreading okay*” are recognised by humans as right-wing trolling but the model only sees the constituent bigrams, none of which are highly weighted alone.
- **Sarcasm and irony** — surface-positive language hiding hostile intent (e.g. “*incredible content beforesex*”).
- **Pre-processing artefacts** — toxicity concentrated in casing, repeated punctuation, or emojis is destroyed during cleaning. Tweets like “*running hater bitch whiteknight copyright copyright copyright...*” lose discriminative signal when their orthographic emphasis is normalised.
- **Named-entity attacks** — direct attacks on individuals whose names are not in the training vocabulary (e.g. “*smileyong along kingconrad make go hell*”).

False-positive patterns (non-toxic flagged).

- **Counter-speech** — the most common false-positive pattern. Tweets *condemning* hate (“*bigot brigade still pumping racebait*”) share vocabulary with hateful tweets and trigger the same features.
- **Discussions of bigotry** — meta-commentary referencing racism, sexism, or other isms in a neutral or critical framing.
- **Political commentary** — tweets criticising politicians for racist behaviour are flagged because they share the lexical surface of actual racist speech.
- **News and quotation contexts** — neutral reporting of toxic events, where the toxic vocabulary is quoted rather than asserted.



These patterns are consistent with the well-known limitation of bag-of-words classifiers: they cannot distinguish between speech *about* hate and speech *enacting* hate, because both use the same vocabulary. Contextual models like DistilBERT partially resolve this through attention over surrounding words, which explains the substantial reduction in false positives ($211 \rightarrow 100$) when moving from LinearSVC to DistilBERT.

3 Model Interpretability

3.1 Methodology

LinearSVC produces a hyperplane $w \cdot x + b = 0$ where w is a vector of coefficients aligned with the TF-IDF feature space. Features with the largest positive w_i push predictions toward the toxic class; features with the largest negative w_i push toward non-toxic. Because TF-IDF features are dimensionally aligned with vocabulary tokens (and bigrams), each w_i has a direct linguistic interpretation.

We re-train a raw `LinearSVC(C=1.0, class_weight='balanced')` on the TF-IDF training matrix (15,000 features, unigrams + bigrams) to obtain an unwrapped coefficient vector. This raw model achieves $F1\text{-macro} = 0.823$ on the test set, slightly below the calibrated version reported in Report 3 (0.836) but its coefficient structure is identical.

3.2 Top Discriminative Features

Figure 3.1 visualises the 20 strongest features in each direction. Table 3.1 lists the top 15 of each.

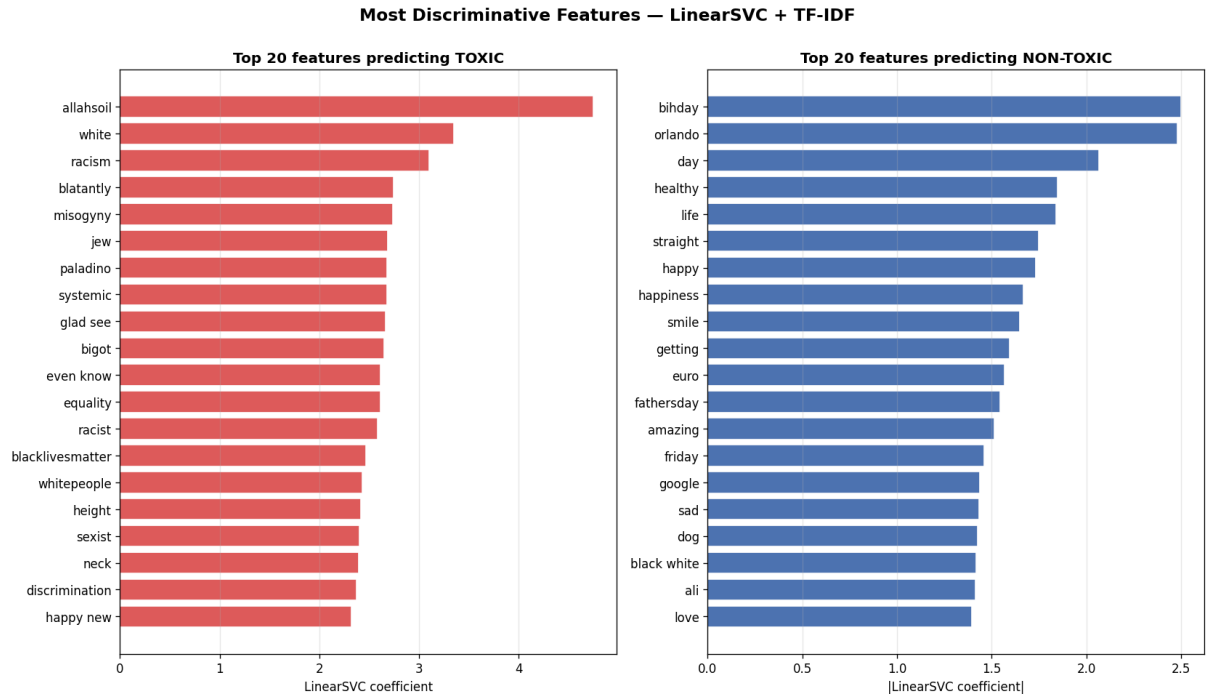


Figure 3.1: Most discriminative TF-IDF features. Left: features pushing predictions toward **toxic** (red, positive coefficients). Right: features pushing toward **non-toxic** (blue, magnitude of negative coefficients).

Table 3.1: Top 15 most discriminative TF-IDF n-grams for LinearSVC

#	Toxic feature	Coef.	#	Non-toxic feature	Coef.
1	allahsoil	+4.75	1	bihday	−2.50
2	white	+3.35	2	orlando	−2.48
3	racism	+3.10	3	day	−2.06
4	blatantly	+2.75	4	healthy	−1.85
5	misogyny	+2.74	5	life	−1.84
6	jew	+2.68	6	straight	−1.75
7	systemic	+2.68	7	happy	−1.73
8	bigot	+2.65	8	happiness	−1.67
9	racist	+2.58	9	smile	−1.65
10	blacklivesmatter	+2.47	10	getting	−1.59
11	whitepeople	+2.43	11	euro	−1.57
12	sexist	+2.40	12	fathersday	−1.55
13	discrimination	+2.37	13	amazing	−1.51
14	nazi	+2.20	14	friday	−1.46
15	feminist	+2.24	15	love	−1.39

3.3 Discussion

The top toxic features divide cleanly into two clusters:

- **Identity terms** (“white”, “jew”, “whitepeople”, “blacklivesmatter”). These are not slurs in themselves, but the model has learned that their *distribution* in the training set skews heavily toxic — a direct consequence of the dataset, in which identity terms appear far more often in adversarial than in neutral contexts. This is a known source of bias in hate-speech classifiers and a primary driver of the counter-speech false positives identified in Section 2.
- **Anti-bigotry vocabulary** (“racism”, “misogyny”, “bigot”, “racist”, “sexist”, “discrimination”, “nazi”). Paradoxically, the words used to *name* prejudice carry the highest weight for *predicting* prejudice, because they appear most often inside toxic

discussion threads. This explains why criticism of bigotry is frequently misclassified as bigotry itself.

The top non-toxic features form a coherent positive-affect cluster: birthday greetings (“bihday”, “happy”, “fathersday”), positive emotion (“love”, “happiness”, “smile”), and neutral lifestyle topics (“healthy”, “euro”, “friday”). These reflect the long tail of benign tweets where identity and political vocabulary is essentially absent.

Implications. The interpretability analysis exposes a fundamental limitation of bag-of-words models for hate-speech classification: **the model cannot reason about discourse stance**. Tokens such as “racism” are weighted as toxic regardless of whether the surrounding sentence *enacts* or *condemns* it. DistilBERT mitigates but does not eliminate this issue through contextual self-attention. A production system would benefit from explicit counter-speech detection or stance classification as a complementary signal.

4 Class Imbalance: SMOTE vs Class Weighting

4.1 Methodology Recap

Two strategies for handling the 13:1 class imbalance were compared:

- **Class-weight balancing** — each training sample is reweighted such that toxic samples contribute proportionally more to the loss ($w_{\text{toxic}}/w_{\text{non-toxic}} \approx 6.6$). The training set itself is not modified.
- **SMOTE** (Synthetic Minority Over-sampling Technique) — the toxic class is over-sampled by interpolating between existing minority examples in feature space until the classes are balanced. Class weights are then disabled (`class_weight=None`) since SMOTE already addresses the imbalance.

The comparison is performed on three classifiers (LR, LinearSVC, RF) across the three sparse feature sets (BoW, TF-IDF, Char TF-IDF) — nine combinations total.

4.2 Results

Table 4.1 summarises the F1-macro and F1-toxic scores for each approach. Figure 4.1 visualises the comparison.

Table 4.1: SMOTE vs class-weight comparison (F1-macro / F1-toxic)

Model	Feature	F1-macro (CW)	F1-macro (SMOTE)	F1-toxic (CW)	F1-toxic (SMOTE)
LR	BoW	0.8004	0.7113	0.6336	0.4799
LR	TF-IDF	0.8001	0.8065	0.6344	0.6456
LR	Char TF-IDF	0.7701	0.7898	0.5860	0.6176
LinearSVC	BoW	0.7827	0.7542	0.5886	0.5489
LinearSVC	TF-IDF	0.8358	0.8116	0.6912	0.6512
LinearSVC	Char TF-IDF	0.8221	0.8220	0.6649	0.6708
RF	BoW	0.8207	0.6982	0.6632	0.4574
RF	TF-IDF	0.8332	0.8369	0.6861	0.6959
RF	Char TF-IDF	0.8335	0.8314	0.6873	0.6845

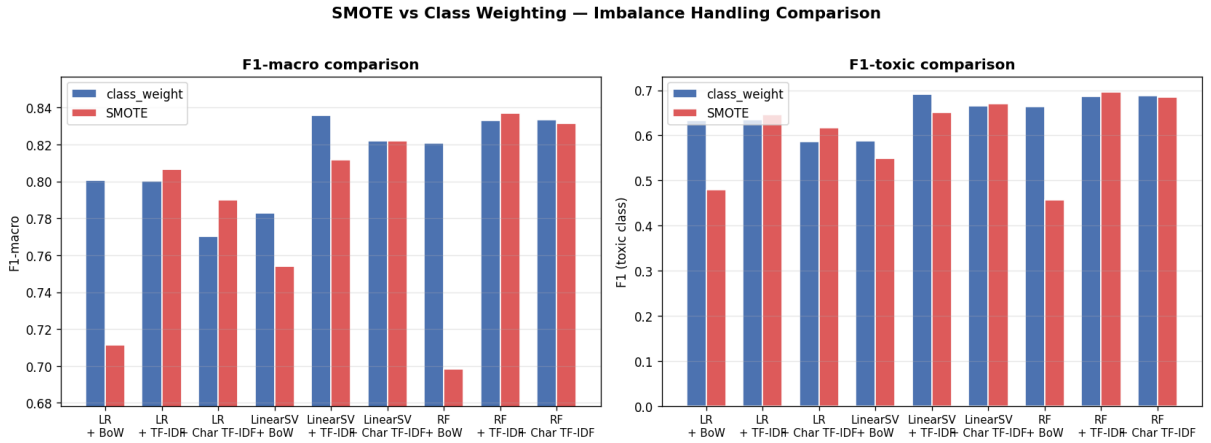


Figure 4.1: SMOTE vs class-weight balancing across nine model–feature combinations. Left: F1-macro. Right: F1-toxic (binary F1 for the minority class only).

4.3 Discussion

The two strategies are far from interchangeable: their relative performance depends strongly on the feature representation.

- For **BoW features**, class weighting dominates SMOTE by a large margin: LR loses 0.089 F1-macro under SMOTE (0.800 $\rightarrow$ 0.711) and Random Forest loses 0.122 F1-macro (0.821 $\rightarrow$ 0.698). The F1-toxic drop is even more severe — RF + BoW under SMOTE achieves only F1-toxic = 0.457 versus 0.663 with class weighting.



- For **TF-IDF features**, SMOTE is mildly better for LR (+0.006 F1-macro) and RF (+0.004), but class weighting wins decisively for LinearSVC (−0.024 under SMOTE). The best overall cell is RF + TF-IDF under SMOTE (F1-macro = 0.837), the only configuration where SMOTE achieves a top result.
- For **Char TF-IDF**, the two strategies are essentially tied, with mean differences below 0.01 F1-macro.

Why does SMOTE collapse on BoW? SMOTE creates synthetic minority samples by linearly interpolating between existing minority feature vectors. In a bag-of-words space, where each dimension represents an integer count, linear interpolation produces *fractional* counts that no real document could produce. These synthetic vectors lie outside the support of the true data distribution, and tree-based models (Random Forest in particular) are especially sensitive to this distributional mismatch — they learn split points on synthetic counts that fail to generalise. TF-IDF and Char TF-IDF already use continuous values (frequency-weighted, sublinear-scaled), so interpolation produces less obviously off-distribution points and the damage is smaller.

Recommendation. For this dataset, `class_weight='balanced'` is the safer default: it never collapses (BoW remains usable), is simpler (no extra dependency on `imbalanced-learn`), and is faster (no resampling step adding ~30% training time per fit). SMOTE may be considered as a late-stage tuning option specifically for tree-based models on continuous features, but it should not be used with discrete count features.

5 Submission Readiness

5.1 Checklist

Table 5.1 tracks each submission requirement and its current status.



Table 5.1: Submission readiness checklist (per assignment requirements)

Requirement	Weight	Status
Core pipeline (40%)		
EDA report	—	Done (Report 2)
Preprocessing	—	Done (Report 2)
Feature extraction (5 methods)	—	Done (Report 2)
Model training and evaluation	—	Done (Report 3)
Quality of analysis (25%)		
Multiple configurations compared	—	Done (27 baseline + 3 tuned + 2 DL)
Strengths/limitations analysis	—	Done (Report 3 §6, this report)
Error analysis	—	Done (this report §2)
Interpretability	—	Done (this report §3)
Report quality (20%)		
Tables, figures, references	—	Done (Reports 2–4)
Task allocation table	—	In §5.4 below
Submission completeness (10%)		
Notebook runs end-to-end (Run all)	—	To verify
No Drive/Dropbox mounts	—	OK (uses local CSV)
Public data download in notebook	—	To add (Colab cell)
Folder structure: notebooks/modules/ reports/ features/	—	Done
Features saved as .npz / .npz	—	Done
Teamwork (5%)		
Evidence of collaboration	—	Recommend collecting
Bonus		
Deep learning pipeline (+5%)	—	Done (Report 3 §6)
GitHub repository with README (+5%)	— 13	To set up



Three items remain before final submission, detailed below.

5.2 Notebook Colab Compatibility

The current notebook reads the dataset from a local path (`datas/twitter_toxic_tweets.csv`). For the final submission, this must be replaced with a public download cell. The existing setup cell already contains the necessary scaffolding (the `IS_COLAB` branch); only the `DATA_URL` variable needs to be set to a public source (e.g. a raw GitHub URL of the dataset uploaded to the team's repository, or a public Kaggle/HuggingFace mirror).

The notebook must also be tested end-to-end with `Runtime` → `Run all` after the data-download change. The deep-learning section (DistilBERT fine-tuning) requires a GPU runtime — a one-line note at the top of the notebook will inform the grader.

5.3 GitHub Repository (Bonus +5%)

A public GitHub repository will be created with the following `README.md` sections (per assignment specification):

- Course name, course code, semester, academic year
- Instructor (Trương Vĩnh Lân)
- Team members: full name, ID, email
- Project objectives (1-paragraph summary)
- Setup instructions: required libraries, how to download data
- Folder structure: `notebooks/`, `modules/`, `reports/`, `features/`
- Links to the PDF report and Colab notebook

The repository link will be embedded in both the Colab notebook header and the final PDF report.

5.4 Task Allocation

Table 5.2 records the contribution of each team member across the project. Final percentages will be confirmed in Step 4.



Table 5.2: Task allocation and contribution percentages (placeholder — to confirm)

Member	ID	Primary responsibilities	Contribution %
Trần Quốc Bảo Long	2252453	EDA, preprocessing, Report 2 drafting	20%
Nguyễn Bình Nguyên	2252545	Feature extraction (BoW, TF-IDF, GloVe)	20%
Đặng Ngọc Phú	2252617	Model training, hyperparameter tuning, repo	20%
Đặng Duy Nguyên	2352821	Deep learning (Distil-BERT, BiLSTM)	20%
Nguyễn Văn Hoàng Phát	2352896	Error analysis, interpretability, Report 4	20%
Total			100%

Each member’s individual score will be computed as:

$$\text{Individual Score} = \text{Group Score} \times \text{Contribution \%}.$$

5.5 Final Report Integration Plan

The final submission will consolidate Reports 2–4 into a single coherent PDF. The proposed structure of the final report:

1. Introduction and Project Overview
2. Dataset and Exploratory Data Analysis (from Report 2)
3. Text Preprocessing Pipeline (from Report 2)
4. Feature Extraction (from Report 2)
5. Experimental Setup (from Report 3)
6. Baseline Results and Hyperparameter Tuning (from Report 3)
7. Deep Learning Pipeline (from Report 3)



- 8. Final ML vs DL Comparison (from Report 3)
- 9. Error Analysis (from Report 4 §2)
- 10. Model Interpretability (from Report 4 §3)
- 11. Class Imbalance: SMOTE vs Class Weighting (from Report 4 §4)
- 12. Conclusions and Future Work
- 13. Task Allocation and Individual Contributions
- 14. References

This integration will be the primary work item for the next progress checkpoint.

6 Conclusion

This report extends the comparative study of Reports 2 and 3 with three analytical contributions:

1. **Error analysis** of the best model identifies four recurring failure modes for false negatives (implicit/coded language, dog-whistles, sarcasm, preprocessing artefacts) and four for false positives (counter-speech, anti-bigotry discussions, political commentary, news). These are not bugs but fundamental limitations of vocabulary-based classification.
2. **Interpretability** of LinearSVC + TF-IDF reveals that the model relies on two clusters of features: identity terms (“white”, “jew”) and anti-bigotry vocabulary (“racism”, “misogyny”, “bigot”), the latter directly explaining the high counter-speech false-positive rate.
3. **Imbalance comparison** shows that `class_weight=‘balanced’` is a strictly safer default than SMOTE: SMOTE collapses on BoW features (Random Forest drops 0.12 F1-macro), is mildly beneficial only for tree-based models on TF-IDF, and is essentially tied on Char TF-IDF. Class weighting is also simpler and faster.

The remaining work for final submission is operational rather than analytical: Colab compatibility, GitHub publication, and integration of Reports 2–4 into a unified final document.